Response to Reviewer (Revision Round 4)

We thank the reviewer for a rigorous and constructive report. The review's central observation — that several interpretations exceeded what a single-host, saturated, single-implementation design identifies, and that this concern has recurred — is correct, and we have addressed it structurally rather than cosmetically. The governing change this round is a **reposition**: the paper is now presented as a *reproducible benchmark instrument and a configuration-specific comparison*, not as a generalized ranking of database libraries. Single host, idiomatic-choice, and the saturated operating point are now **declared scope**, and the harness, its controls, and a benchmarking-pitfalls checklist are the primary contributions. We also ran the feasible experiments the review pointed toward. Section and table references are to the revised submission build; effective length is recorded in `word-count.pdf`.

The reposition (addresses the recurring "claims exceed design")

- **Title, Objective, RQs, contributions recast.** The title now reads *"A

Reproducible Benchmark . . . A Configuration-Specific Comparison on PostgreSQL and MySQL." *The Objective leads with the harness ("the harness, not a generalized ranking, is the contribution"). Each RQ is scoped to "the benchmarked implementations under the specified operating point." The contributions list the instrument, the design, and the checklist as primary**; the empirical findings are labelled configuration-specific.

- **We propagated the fixes uniformly.** The recurrence was partly mechanical: prior

rounds fixed a phrase in one place but not others, so the paper contradicted itself. We swept every occurrence. There are now **zero** instances of "independent replicate(s)" (the runs are "repeated randomized-block runs"; the paper already conceded non-independence, so this removes a self-contradiction), the primary p99 is called "saturated client-observed response-time p99" throughout, and the same-SQL control is described only as a compound intervention.

Major concerns

§6.1 — saturated p99 is a saturation outcome, not intrinsic tail. Agreed. It is now consistently named a saturated client-observed response-time p99 at the stated operating point, and we added a **utilization-controlled open-loop experiment**: each layer is offered 50/70/85/95 % of its own saturating throughput, replicated, on both engines. At 50 % of each layer's own capacity every layer holds p99 near 2–4 ms on both engines, and the large saturated gap (native driver 20 ms versus the slowest ORM 106 ms) dissolves — confirming the saturated ladder was tracking capacity/queueing, not intrinsic tail latency; the tail rises only as a layer nears its own capacity. The utilization results are now at least as prominent as the saturated p99.

§6.2 — same-SQL does not identify a mechanism. Agreed and corrected everywhere. The control is now described strictly as a compound intervention that changes query formulation, API, protocol, statement preparation, round-trip structure, and mapping *together* and isolates no single factor; the residual is not called "raw-execution cost" unqualified. A new **controlled/uncontrolled factor table** (now Supplement Table S27) makes explicit what the control holds constant and what it changes together.

§6.3 — "idiomatic" is subjective. We added a **pre-specified selection protocol**, fixed before measurement: the eager-loading API each library's official documentation presents first for the pinned version, with logging/hooks/validation off. The exact API, **documentation page, and pinned version** for each layer are tabulated (`METHODOLOGY.md`, Supplement Table S16). We did not invite maintainers to review the adapters; that is now disclosed as an explicit

limitation. The alternative loading strategy is measured where it is a byte-identical drop-in; the native, query-builder, and Prisma layers have no second loading strategy to offer (their SQL is fixed), so "strongest alternative for every layer" is satisfied where feasible.

§6.4 — equal pool ≠ equal resource. We renamed the estimand "equal configured connection limit" and added a **per-layer pool-size frontier** on both engines (throughput versus pool size, 1–50) so the fixed-pool choice is shown not to drive the ranking.

§6.5 — single-host interference. We do not have a second machine, so a separate-host / remote-database topology is named as a scoped future use of the instrument rather than claimed. The feasible test of the reviewer's specific worry — that Prisma's multi-core engine looks competitive only because the one-core layers leave cores idle — is a **multi-worker (node cluster) experiment**: each layer is scaled to 1/2/4 workers so every layer can use the spare cores. The result confirms the reviewer's intuition: at one worker the native driver and Prisma are near parity, but at four workers the native driver overtakes Prisma by 1.6× on PostgreSQL and 1.4× on MySQL, because the single-pool layers scale on the cores Prisma's engine already consumed. Prisma's throughput parity is therefore specific to a deployment that leaves cores idle. The single-host topology, and its consequences for the CPU comparison, are disclosed prominently.

§6.6 — "25 independent replicates." Corrected to "25 repeated runs" throughout — the body, the main-text and supplement table captions, the generic build, and the table-generating scripts (so a re-run of the pipeline is also clean). The manuscript's own non-independence paragraphs are retained.

§6.7 — narrow workload / broad recommendations. Recommendations are now scoped to "the benchmarked implementations in this configuration, workload, and host," and the unmeasured productivity/type-safety trade-off is flagged as unmeasured. We added a **mixed read/write workload** (interleaved deep-fetch reads and inserts at documented mixes) and a **60–120 s longer-run sensitivity** subset. Both preserve the read ordering, and the 90 s runs reproduce the 12 s medians within 1 %.

§6.8 — selective/post-hoc statistics. Every primary layer×engine×pattern cell now carries a **95 % bootstrap confidence interval on throughput** in the tables (not only medians and selected adjacent tests). The single-run open-loop and three-run fan-out are **replicated** (open-loop to five, fan-out to eight runs) and the fan-out, durability, loading-strategy, and open-loop analyses are explicitly labelled exploratory. The adjacent-rank tests were already labelled post-hoc.

§6.9 — the interaction p is uninformative. We now read the interaction through its **effect size** (per-layer cross-engine throughput ratio, narrow 1.0–1.6 on reads, wide 2.4–7.7 on writes) and note that the floor p conveys little about magnitude; a **per-layer PG-vs-MySQL rank table** now backs the correlations (Prisma flips from rank 1 on PostgreSQL to rank 6 on MySQL for inserts).

§6.10 — novelty. We call the literature review a *structured related-work search* (not systematic), scope the novelty to the searched databases and dates, and archive the search strings.

§11 — length and repetition. Addressed. The five conclusions the review found restated across sections are now stated once, in the results, and cross-referenced elsewhere; the secondary access patterns (point read, keyset range scan) and every secondary experiment are now in the numbered online supplement, leaving only the primary factorial, the same-SQL and equal-CPU controls, and the utilization result as main-text floats; the controlled/uncontrolled factor table also moved to the supplement (Table S27); and five peripheral citations (general NoSQL and OLTP-benchmark context) were dropped, taking the reference list from 70 to 65. Counting

references and floats per the IST guide, the main text is now 14,848 words, within the 15,000-word limit (`word-count.pdf`).

What was already correct (verified, not changed)

We note, in case it saves the reviewer time, that several items the review asked us to check are already implemented as recommended:

- **p99 test unit** is the run-level statistic (25 per cell), never the individual

request; the paper and code (`stats2.mjs`) both use run-level values.

- **Bootstrap** is a **percentile** bootstrap resampling **replicate indices**

(preserving pairing), from a **fixed seed** (`mulberry32(0x57a75)`); this is now stated in the methodology.

- **Docker images** are pinned by immutable `@sha256` digest; **dependencies** by a

committed lockfile.

- All six **declarations** (structured abstract, CRediT, competing interest, funding,

data availability with the Zenodo DOI, generative-AI) are present.

- The **author name** is consistent ("Miotk") throughout; there is no "Miotka".

Point-by-point answers to the reviewer's questions

1. **Idiomatic selection rule?** Pre-specified before measurement: the first

eager-loading API in each library's official guide for the pinned version; documented with page and version (Supplement Table S16, `METHODOLOGY.md`).

1. **Maintainer/expert review?** No; disclosed as a limitation.
2. **Equivalent tracking/hooks/validation/lifecycle?** Library defaults, disabled or

request-scoped on the read path; documented per adapter.

1. **Same-SQL mappers identical machine code?** Source-level equivalent, not identical

code paths; this is exactly why the control is a compound intervention.

1. **Prepared-statement caches over fresh processes?** A fresh process per run means a

cold statement cache each run; the tuned baselines reuse within a run.

1. **Bootstrap resampling unit?** Replicate index (25 run-level values), seeded.
2. **p99 tests on 25 run-level values?** Yes.
3. **Why saturated p99 primary vs replicated open-loop at controlled utilization?**

Both are now reported; the utilization-controlled open-loop is the near-intrinsic latency evidence.

1. **Processes CPU-pinned?** DB/server/generator share the host; the equal-CPU control

pins the server; single-host is disclosed.

1. **Steal time / frequency / neighbours?** Uncontrolled on the virtualized host;

disclosed; the drift, post-restart, and longer-run checks bound their effect.

1. **Load generator co-located every run?** Yes (single host); disclosed.

2. **Buffer pools / OS caches between cells?** Warm by design; randomized order and fresh boot; the post-restart check confirms the cold-start ranking.

1. **Exact IST word count?** Recomputed (`word-count.pdf`).
2. **Zenodo archive = submitted version?** Yes; the v1.4.0 tarball is ‘git archive HEAD‘ of the tagged release.

1. **Fan-out "robust" with three runs?** The reviewer was right to doubt it. On replication (all layers, eight runs, both engines) the spread is a **roughly constant** multiplier across 0–500 comments, not the growing one the three-run sweep suggested; we removed the "growing with graph size" claim from the abstract, intro, results, discussion, conclusion, and threats, and report the fan-out as exploratory.

1. **Uncertainty for the MySQL commit-path percentages?** Reported from the wait-event replicates.

1. **Stable public versions + immutable ids?** Digest-pinned images + lockfile.
2. **Miotk or Miotka?** Miotk everywhere (verified).
3. **AI generate/modify adapters or stats code?** Assisted; verified by the byte-identical cross-check and unit tests; disclosed.

1. **Raw logs reconstruct failed/excluded runs (knex n=24)?** Yes; raw JSON, partial files, and status logs are archived.

We believe the repositioned paper claims exactly what a single-host benchmark identifies, backed by new experiments and a fully documented, provenance-tracked instrument, and we thank the reviewer for pressing it to this point.